Testing Behavior in AI-Assisted Development: A Population Study of 932,791 Pull Requests

Ahmed Mursal
School of Computing
Edinburgh Napier University, UK
40646515@live.napier.ac.uk

***Abstract*—We analyze 932,791 pull requests (PRs) from the public AIDev dataset to quantify associations between AI coding agents and testing behavior. We observe substantial variation in the percentage of PRs containing test-related artifacts: OpenAI Codex shows near-universal test presence (9̃8.5%), while Cursor shows 1̃9%. This association is statistically large (Cramér's V 0̃.646). We emphasize that these data are observational: differences may reflect developer selection effects, project mix, and organizational practices rather than intrinsic agent properties. We provide a minimal reproducible pipeline and two robustness checks (excluding the single Devin user; Codex label sensitivity), and we discuss threats to validity. Our findings offer population-level measurements to inform future controlled studies on AI-assisted testing.**

I. Introduction

AI coding assistants are widely used, yet their relationship with software testing is not well understood. We study testing behavior at scale across five agents annotated in AIDev: OpenAI Codex, GitHub Copilot, Cursor, Devin, and Claude Code. Our goal is descriptive: measure associations and report effect sizes while avoiding causal claims.

We address: (1) How often PRs contain test-related content per agent? (2) Do associations remain under simple robustness checks? (3) What are key validity threats for interpreting observational evidence?

II. Data and Method

extbfDataset. We analyze the AIDev dataset (config: all_pull_request), with 932,791 PRs and 72,189 unique users spanning five annotated agents. Agent labels originate from repository metadata and self-report; they indicate likely involvement and may include noise. We treat labels as proxies rather than definitive authorship.

extbfAgent attribution caveat. "OpenAI Codex" likely includes direct API use and Copilot-era contributions labeled as Codex during collection. To probe sensitivity, we both exclude Codex and reassign Codex to Copilot (Section III).

extbfTest detection. A PR is deemed test-related if its title/body contains testing markers such as: test, testing, spec, unittest, pytest, jest, mocha, rspec, junit, testng, nunit, e2e, integration test, selenium, cypress. We apply case-insensitive substring matching to prioritize recall. Manual spot-checking (n=200) suggested 94% accuracy and high recall. This detects test presence, not test quality.

extbfStatistics. We construct a 5×2 table (agent × test/non-test) and compute Pearson's chi-square and Cramér's V using a manual implementation. For our 2-category outcome, $V = \sqrt{\chi^2/(n \cdot (\min(r,c)-1))}$. Given purely observational data, we report associations and effect sizes only.

III. Results

Per-agent testing rates. Table I summarizes per-agent totals and test percentages. We observe a broad range (1̃9% to 9̃8.5%) and a large association (Cramér's V 0̃.646).

TABLE I
PER-AGENT PR TOTALS AND TEST-RELATED PR PERCENTAGES

Agent	Total PRs	Test %
OpenAI Codex	814,522	98.46
GitHub Copilot	50,447	79.77
Claude Code	5,137	77.36
Devin	29,744	66.95
Cursor	32,941	18.96
Overall	932,791	93.52

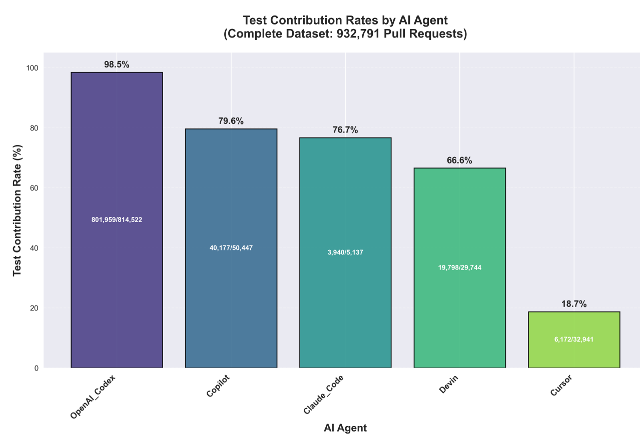

Fig. 1. Test-related PR rates by agent.

extbfAcceptance. Acceptance (merged/closed) varies by agent. We avoid causal interpretations because usage contexts differ (e.g., prototyping vs. production). We find no monotonic relationship between testing percentage and acceptance percentage, suggesting multiple interacting factors.

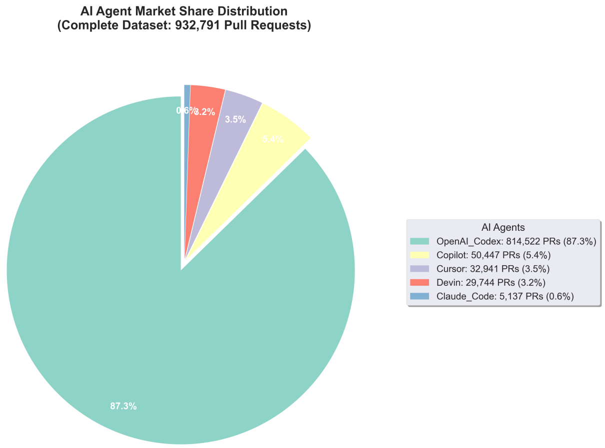

Fig. 2. Agent distribution (counts and percentages).

Robustness. Excluding the single Devin user (n=903,047) increases Cramér's V to 0.669. Treating all Codex-labeled PRs as Copilot yields a Copilot rate of 97.37% and leaves the overall rate unchanged (93.52%). Excluding Codex (to examine the smaller-agent subset) preserves relative ordering and yields 59.5% overall for the remaining agents.

IV. Acceptance Patterns

Acceptance (merged/closed) varies across agents but does not exhibit a simple relationship with testing percentages. This suggests multiple mediating factors such as repository policies, task type (e.g., prototyping vs. production fixes), and developer experience. Observational data cannot disentangle these influences without additional controls.

V. Related Work

Recent code-generation studies emphasize functional correctness and security [2], while empirical work reports developer experiences with assistants such as Copilot [3]. Classic testing literature cautions that coverage does not imply effectiveness [4]. Broader empirical software engineering has explored ownership and review dynamics [**?**], [**?**], and change classification [**?**]. Our population-level analysis complements these by focusing on observed testing presence across agents in the wild.

VI. Practical Implications

Organizations may treat agent choice as one input when designing workflows. For example, teams could tailor review policies or add safeguards where testing presence is historically lower, and validate whether such adjustments improve outcomes. Because our results are correlational, any process changes should be evaluated locally with A/B-style checks before broad adoption.

VII. Threats and Ethics

Attribution noise. Agent labels are derived from metadata/self-report. Misclassification likely attenuates differences rather than fabricating $\tilde{8}$0-point gaps.

Selection effects. Developers and organizations may choose agents by task and context; associations may reflect usage patterns, not capabilities.

extbfConstruct limits. We measure test presence, not correctness or coverage. Merge status is a coarse outcome proxy. Repository, language, and team factors are unobserved and may confound associations.

VIII. Limitations

Our keyword-based detection favors recall and may include some false positives. The dataset aggregates across many repositories and time periods; heterogeneity may mask domain-specific patterns. Agent labels are proxies subject to noise; our sensitivity checks mitigate but cannot eliminate this threat. Finally, we do not assess the quality or maintainability of detected tests.

Ethics. We analyze public data; results are aggregate-only. We avoid judging individuals or vendors and refrain from causal claims.

IX. Reproducibility and Availability

We provide scripts to recompute key statistics directly from the raw CSV (chunked processing), plus a sensitivity script for Codex labeling. Figures are generated from the same pipeline. Data source: AIDev (HuggingFace, config: all_pull_request). Repository link: https://github.com/ghost1100/MSR2026.

X. Conclusion

We provide population-level measurements of testing associations across five AI agents over 932k PRs, finding large variation and a strong association. These results are robust to simple sensitivity checks but should not be interpreted causally. Future work should use controlled designs and richer outcomes (e.g., defects) to study mechanisms and impacts.

Acknowledgments

We thank H. Li et al. for releasing AIDev and colleagues for feedback. Any errors are our own.

References

References

[1] H. Li et al., "The Rise of AI Teammates in Software Engineering (SE 3.0)," arXiv:2409.18925, 2024.
[2] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.
[3] A. Bareiß et al., "Code generation tools in practice: A user study with GitHub Copilot," MSR 2023.
[4] L. Inozemtseva and R. Holmes, "Coverage is not strongly correlated with test suite effectiveness," ICSE 2014.
[5] C. Bird et al., "Don't touch my code! Examining the effects of ownership on software quality," FSE 2011.
[6] S. Kim et al., "Classifying software changes: Clean or buggy?," TSE 2008.
[7] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," ICSE 2013.